RAPPORT DE TRAVAUX PRATIQUES

المدرسة الوطنية للعلوم التطبيقية - بني ملال

Ecole Nationale des Sciences Appliquées - Béni Mellal

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BÉNI MELLAL

TP1 – Application Monolithique Spring Boot

Réalisé par : EL MELIANI Ammar
Encadrant : Pr. EL BAGHAZAOUI

Filière : **IACS-S2**
Année universitaire : 2025/2026

Lien GITHUB :

https://github.com/EL-Star456/TP_Rapport_Log.git

TP 1 - Application Monolithique avec Spring Boot

1. Objectifs du TP

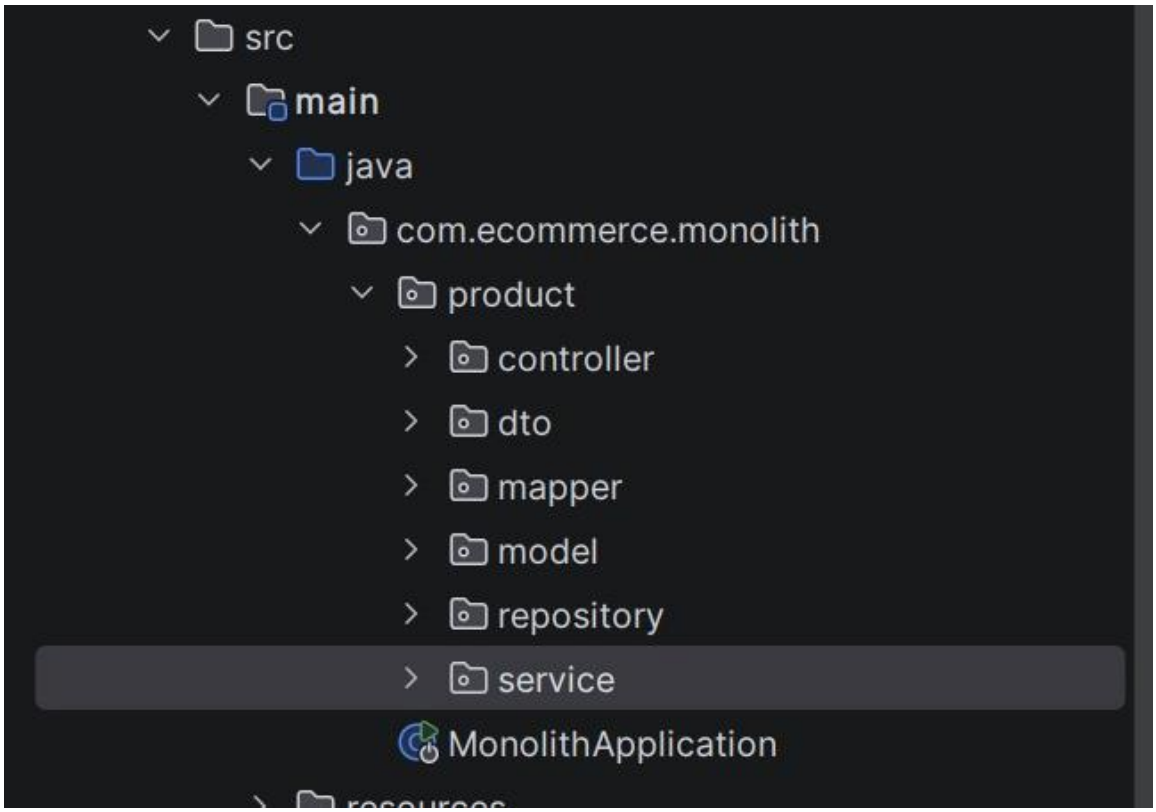
- **Créer une application Spring Boot monolithique simple**
- **Développer une API REST CRUD complète**
- **Comprendre l'architecture en couches (Entity, Repository, Service, Controller)**
- **Découvrir JPA et Hibernate pour la persistance des données**
- **Identifier les avantages et limites d'un monolithe**

2. Architecture du Projet

L'application suit une architecture en couches (Layered Architecture) classique, composée de quatre couches distinctes :

2.1 Structure des packages

```
com.ecommerce.monolith
  product/
    controller/    -> ProductController.java
    service/       -> ProductService.java
    repository/    -> ProductRepository.java
    model/         -> Product.java, Category.java
```



2.2 Description des couches

Controller : Point d'entree de l'API REST. Recoit les requetes HTTP (GET, POST, PUT, DELETE) et retourne des reponses JSON avec les codes HTTP appropries (200, 201, 204).

Service : Contient la logique metier de l'application. Fait le lien entre le Controller et le Repository. Annote avec @Service et @Transactional pour la gestion automatique des transactions.

Repository : Interface d'acces a la base de donnees. Etend JpaRepository qui genere automatiquement toutes les requetes SQL (SELECT, INSERT, UPDATE, DELETE) sans ecrire une seule ligne de SQL.

Model (Entity) : Representation des donnees en Java. Correspond directement aux tables PostgreSQL via les annotations JPA (@Entity, @Table, @Column).

3. Implémentation

3.1 Entité Product

L'entite Product represente un produit dans la base de donnees. Elle utilise les annotations JPA pour le mapping objet-relationnel et les annotations Lombok pour reduire le code repetitif.

3.2 Repository

Le repository etend JpaRepository<Product, Long> qui fournit automatiquement les methodes CRUD. Une methode de recherche par nom est ajoutee pour illustrer les query methods de Spring Data.

3.3 Service

Le service contient la logique metier avec la gestion des transactions. L'annotation @Transactional(readOnly = true) optimise les requetes de lecture, et @Transactional gere automatiquement les commits et rollbacks.

3.4 Controller

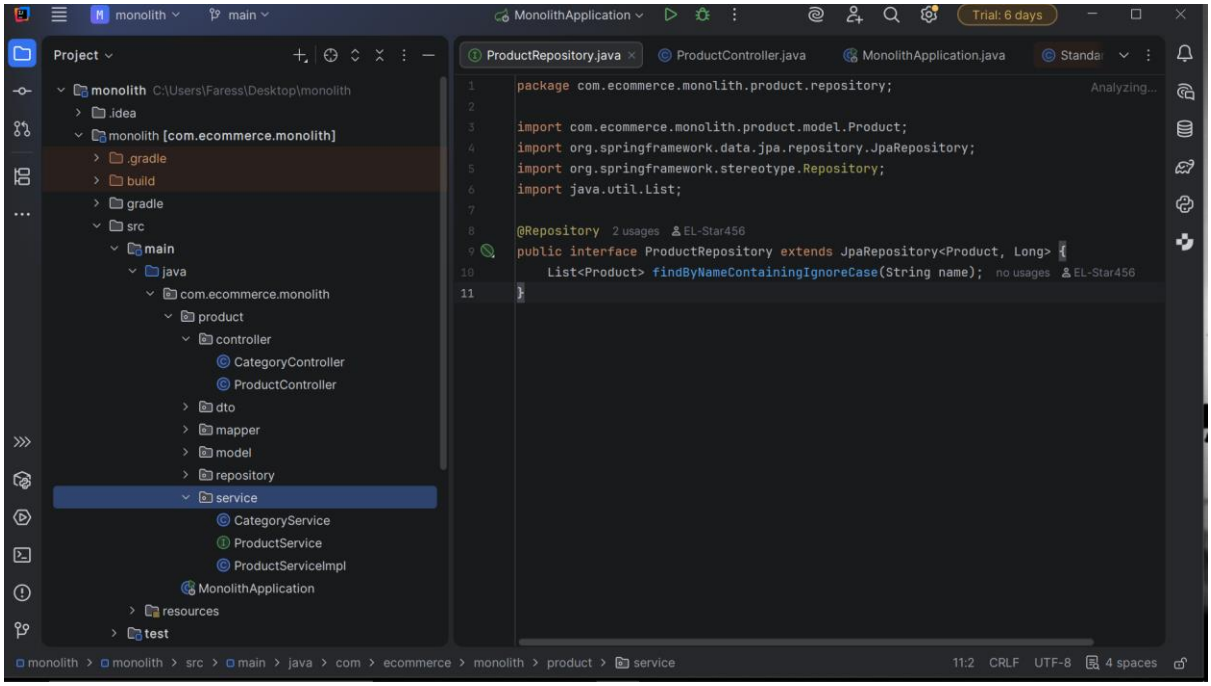
Le controller expose les endpoints REST avec les annotations Spring MVC. Chaque methode HTTP correspond a une operation CRUD :

Methode HTTP	URL	Description	Code Reponse
GET	/api/products	Lister tous les produits	200 OK
GET	/api/products/{id}	Obtenir un produit par ID	200 OK
POST	/api/products	Creer un nouveau produit	201 Created
PUT	/api/products/{id}	Modifier un produit	200 OK
DELETE	/api/products/{id}	Supprimer un produit	204 No Content

4. Exercice - Entite Category

En bonus, une entite Category a ete ajoutee avec une relation ManyToOne avec Product. Cela illustre les relations entre entites JPA et la creation automatique des tables avec les contraintes de cles etrangeres.

Hibernate a genere automatiquement la table categories et la cle etrangere category_id dans la table products, sans aucune intervention manuelle en SQL.



6. Tests avec Postman

Les differents endpoints ont ete testes avec Postman. Voici les resultats obtenus :

Test	Methode	URL	Resultat
Creer un produit	POST	/api/products	201 Created - OK
Lister les produits	GET	/api/products	200 OK - Liste vide puis avec donnees
Modifier un produit	PUT	/api/products/1	200 OK - Produit mis a jour
Supprimer un produit	DELETE	/api/products/1	204 No Content - OK
Creer une categorie	POST	/api/categories	201 Created - OK
Produit avec categorie	POST	/api/products	201 Created avec category_id

POST http://localhost:8080/ GET http://localhost:8080/a POST http://localhost:8080/ No environ

http://localhost:8080/api/products Save

POST http://localhost:8080/api/products Send

Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
{
    "name": "Laptop Dell",
    "description": "Ordinateur portable",
    "price": 1299.99,
    "stock": 10
}
```

Body 201 Created 968 ms 261 B

Pretty Raw Preview Visualize JSON

```
{
    "id": 4,
    "name": "Laptop Dell",
    "description": "Ordinateur portable",
    "price": 1299.99,
    "stock": 10
}
```

Body 200 OK 311 ms 444 B

Pretty Raw Preview Visualize JSON

```
[
    {
        "id": 2,
        "name": "Laptop Dell",
        "description": "Ordinateur portable",
        "price": 1299.99,
        "stock": 10
    },
    {
        "id": 3,
        "name": "Laptop Dell",
        "description": "Ordinateur portable",
        "price": 1299.99,
        "stock": 10
    },
```

7. Conclusion TP1:

Le TP1 a permis de construire une API REST Spring Boot complete avec connexion PostgreSQL. L'architecture en couches (Controller, Service, Repository, Entity) est le standard professionnel utilise dans l'industrie. La puissance de Spring Boot reside dans sa capacite a generer automatiquement le code repetitif (requetes SQL, endpoints CRUD) grace a Spring Data JPA et aux annotations.

Les limites du monolithe simple sont visibles : tout le code est dans un seul module, ce qui devient difficile a maintenir quand le projet grandit. C'est pourquoi le TP2 introduit le concept de monolithe modulaire.

TP 2 - Monolithe Modulaire

1. Objectifs du TP

- **Restructurer le monolithe en modules métier distincts**
- **Définir des boundariès claires entre domaines**
- **Implémenter les design patterns DTO, Mapper et Service Layer**
- **Utiliser MapStruct pour le mapping automatique Entity <-> DTO**
- **Préparer le terrain pour une future migration vers les microservices**

2. Nouveaux Concepts Introduits

2.1 DTO (Data Transfer Object)

Un DTO est un objet intermediaire utilise pour transferer des donnees entre les couches de l'application. Au lieu d'exposer directement l'entite JPA au client, on cree un DTO qui contient uniquement les champs necessaires. Cela apporte plusieurs avantages :

- **Securite :** on ne expose pas les details internes de la base de donnees
- **Flexibilite** : le DTO peut avoir une structure differente de l'entite
- **Versioning** : on peut modifier l'entite sans casser l'API publique
- Performance : on peut limiter les donnees transferees sur le reseau

2.2 MapStruct

MapStruct est un generateur de code Java qui cree automatiquement le code de conversion entre les entites et les DTOs a la compilation. Il elimine le code repetitif de mapping manuel et offre de meilleures performances que les mappers par reflexion comme ModelMapper.

2.3 Interface + Implementation (Service Layer Pattern)

Le Service Layer Pattern separe le contrat (interface) de l'implementation. Cela permet de :

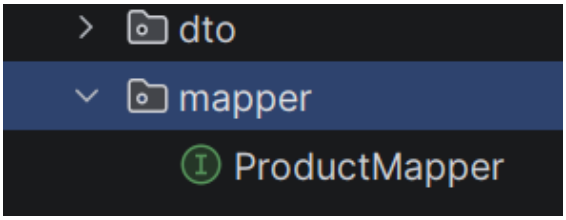
- Faciliter les tests unitaires (on peut mocker l'interface)
- Respecter le principe SOLID (Open/Closed Principle)
- Faciliter le remplacement de l'implementation sans changer le code appelant

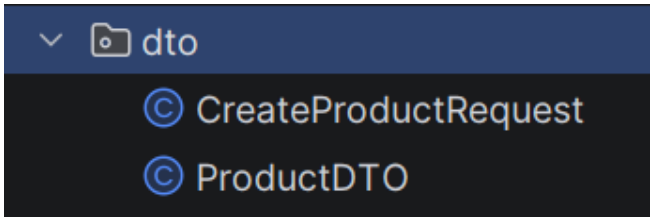
3. Nouvelle Architecture

La structure du projet est reorganisee pour mieux separer les responsabilites :

```
com.ecommerce.monolith
  product/
    controller/  -> ProductController.java
    service/     -> ProductService.java (interface)
                 -> ProductServiceImpl.java (implementation)
```

```
repository/   -> ProductRepository.java
model/        -> Product.java
dto/          -> ProductDTO.java
              -> CreateProductRequest.java
mapper/       -> ProductMapper.java
```





4. Implementation des DTOs

4.1 ProductDTO

Le ProductDTO est utilise pour les reponses de l'API. Il contient tous les champs du produit qui doivent etre retournes au client.

4.2 CreateProductRequest

Le CreateProductRequest est utilise pour les requetes de creation et de modification. Il contient les validations necessaires et n'expose pas le champ id (genere automatiquement).

5. Refactorisation du Service

Le service est separe en deux fichiers : l'interface ProductService qui definit le contrat, et ProductServiceImpl qui contient l'implementation concrete. Le mapper est injecte dans le service via l'injection de dependances de Spring.

Classe/Interface	Role
ProductService (interface)	Definit les methodes disponibles - contrat public
ProductServiceImpl (classe)	Implementation concrete avec logique metier
ProductMapper (interface)	Conversion automatique Entity <-> DTO par MapStruct
ProductDTO	Objet de reponse envoye au client
CreateProductRequest	Objet de requete recu du client avec validation

6. Configuration MapStruct

MapStruct est ajoute dans build.gradle avec trois dependances : la librairie principale, le processeur d'annotations qui genere le code, et le binding avec Lombok pour assurer la compatibilite.

7. Flux d'une Requete dans le TP2

Le flux complet d'une requete POST /api/products dans le TP2 est le suivant :

Etape	Composant	Action
1	Client (Postman)	Envoie POST /api/products avec JSON CreateProductRequest
2	ProductController	Recoit la requete, valide (@Valid), appelle productService.createProduct()
3	ProductServiceImpl	Recoit CreateProductRequest, appelle mapper.toEntity()
4	ProductMapper	Convertit CreateProductRequest -> Product (genere par MapStruct)
5	ProductRepository	Sauvegarde le Product dans PostgreSQL via repository.save()
6	ProductServiceImpl	Recoit le Product sauvegarde, appelle mapper.toDTO()
7	ProductMapper	Convertit Product -> ProductDTO
8	ProductController	Retourne ResponseEntity avec ProductDTO et statut 201 Created
9	Client (Postman)	Recoit la réponse JSON avec le ProductDTO

8. Conclusion TP2 :

Le TP2 a introduit les concepts professionnels du développement Spring Boot : le pattern DTO pour la securite et la flexibilite de l'API, MapStruct pour le mapping automatique, et le Service Layer Pattern pour la testabilite et la maintenabilite.

La restructuration en modules distincts avec des boundaries claires est la base de toute architecture logicielle professionnelle. Cette organisation facilite le travail en equipe, les tests unitaires, et prepare directement la migration vers les microservices.

Les deux TPs ensemble forment une progression logique : du monolithe simple (TP1) vers le monolithe bien structure (TP2), avant la migration complete vers les microservices (TP suivant). Cette demarche incrementale est la meilleure approche pour comprendre les architectures modernes.

TP 3 – Architecture distribuée

1. Objectifs du TP

Le TP3 simule un vrai système e-commerce distribué avec 6 composants qui communiquent ensemble :

Postman

Le client qui envoie les requetes HTTP — comme toi qui passes une commande sur un site.

Nginx :80

Le load balancer — comme un receptionniste qui distribue les appels entre plusieurs employes.

Node.js x2

2 serveurs applicatifs (port 3001 et 3002) — la logique metier, traitement des commandes

MongoDB

Base de donnees NoSQL — stocke toutes les commandes de facon persistante

Redis

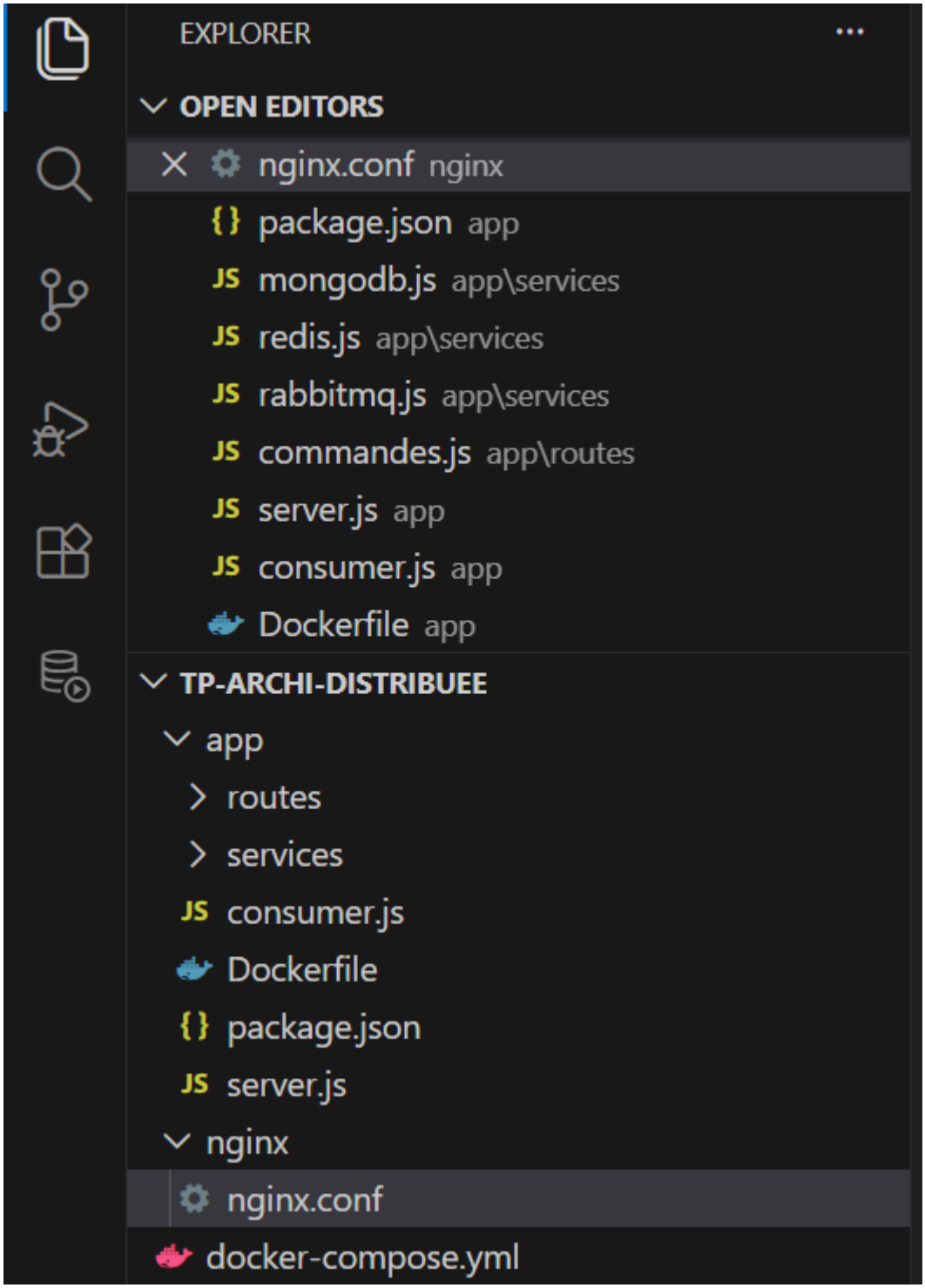
Cache memoire — evite de relire MongoDB a chaque requete GET (ultra rapide)

RabbitMQ

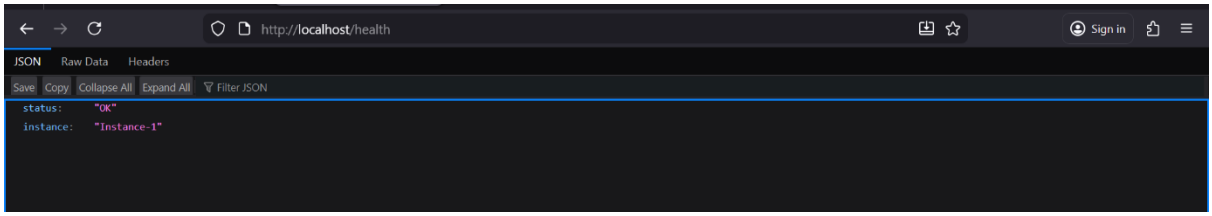
Bus de messages — envoie une notification apres chaque commande (comme un email de confirmation).

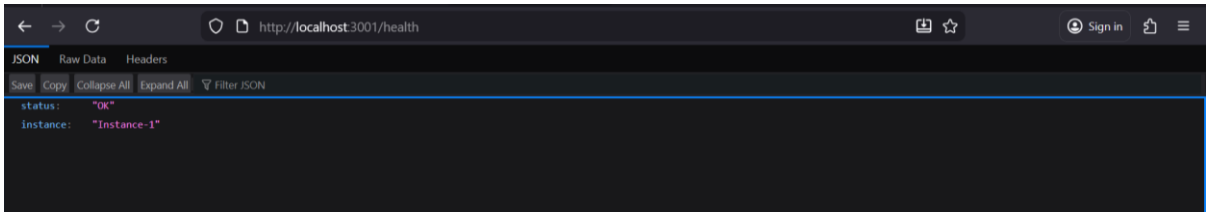
```
[+] Running 8/8
 ✓ Network tp-archi-distribuee_default      Created                    0.2s
 ✓ Volume "tp-archi-distribuee_mongo_data"  Created                    0.0s
 ✓ Container rabbitmq                       Created                    0.3s
 ✓ Container redis                          Created                    0.3s
 ✓ Container mongodb                        Created                    0.4s
 ✓ Container app2                           Created                    0.3s
 ✓ Container app1                           Created                    0.3s
 ✓ Container nginx                          Created                    0.1s
```

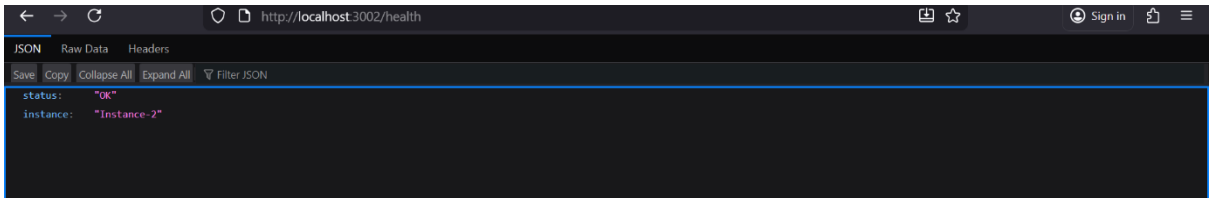
2. Architecture



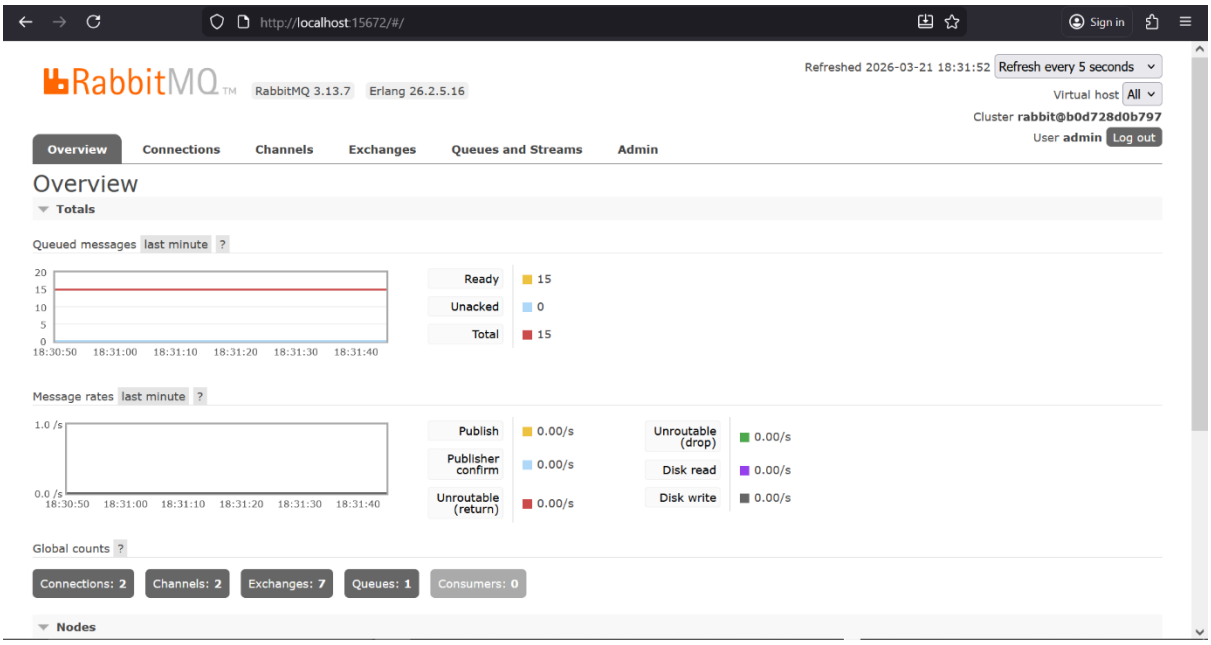
3. URLs


http://localhost/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-1"


http://localhost:3001/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-1"


http://localhost:3002/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-2"

http://localhost:15672
RabbitMQ
Username:
Password:
Login


http://localhost:15672/#/
RabbitMQ
RabbitMQ 3.13.7
Erlang 26.2.5.16
Refreshed 2026-03-21 18:31:52
Refresh every 5 seconds
Virtual host All
Cluster rabbit@b0d728d0b797
User admin
Log out
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Overview
Totals
Queued messages last minute ?
Ready 15
Unacked 0
Total 15
Message rates last minute ?
Publish 0.00/s
Publisher confirm 0.00/s
Unroutable (return) 0.00/s
Unroutable (drop) 0.00/s
Disk read 0.00/s
Disk write 0.00/s
Global counts ?
Connections: 2
Channels: 2
Exchanges: 7
Queues: 1
Consumers: 0
Nodes

4.Test avec postman

POST http://localhost/commandes Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2 "produit" : "Laptop Dell XPS",
3 "quantite" : 2,
4 "client" : "Ahmed Benali"
5 }
```

Body 201 Created 27 ms 472 B

{} JSON Preview Visualize

```
1 {
2     "message": "Commande créée",
3     "commande": {
4         "produit": "Laptop Dell XPS",
5         "quantite": 2,
6         "client": "Ahmed Benali",
7         "statut": "en_attente",
8         "_id": "69bedda74b37b8e8e952ca46",
9         "date": "2026-03-21T18:04:23.723Z",
```

http://localhost/commandes Save Share

GET http://localhost/commandes Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2 "produit" : "Laptop Dell XPS",
3 "quantite" : 2,
4 "client" : "Ahmed Benali"
5 }
```

Body 200 OK 58 ms 764 B

{} JSON Preview Visualize

```
2     "source": "🗄 MongoDB",
3     "traitee_par": "Instance-2",
4     "commandes": [
5         {
6             "_id": "69bedda74b37b8e8e952ca46",
7             "produit": "Laptop Dell XPS",
8             "quantite": 2,
9             "client": "Ahmed Benali",
10            "statut": "en_attente",
11            "date": "2026-03-21T18:04:23.723Z",
12            "__v": 0
13        },
14        {
15            "_id": "69bedd523a770e92ee291a47",
16            "produit": "Laptop",
17            "quantite": 1
```

Body 200 OK 18 ms 2.58 KB

{ } JSON Preview Visualize


```
{
    "source": "⚡ CACHE Redis",
    "traitee_par": "Instance-2",
    "commandes": [
        {
            "_id": "69beded33a770e92ee291a54",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-21T18:09:23.169Z",
            "__v": 0
        },
        {
            "_id": "69bedeca4b37b8e8e952ca52",
```

```
            "_id": "69bedec53a770e92ee291a52",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-21T18:09:09.600Z",
            "__v": 0
        },
```


5. Questions

Q1/Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

Le composant qui reçoit les requêtes en premier est le Load Balancer (ici Nginx). Il agit comme une porte d'entrée unique pour le système. L'algorithme utilisé est le Round-robin. Il distribue les requêtes de manière équitable et séquentielle : la première requête va vers `app1`, la deuxième vers `app2`, la troisième revient à `app1`, et ainsi de suite.

Q2/Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Si `app1` tombe en panne, Nginx continue de fonctionner normalement. Pourquoi ? Parce que Nginx est un composant indépendant des serveurs applicatifs. Sa configuration lui permet de détecter qu'une instance ne répond plus (mécanisme de *Health Check*). Il retire alors temporairement `app1` de sa liste et redirige tout le trafic vers les instances saines (comme `app2`). C'est le principe de la Haute Disponibilité.

Q3/Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET

- **1er GET (Source MongoDB) : La requête est plus lente car l'application doit lire les données sur le disque dur via MongoDB. Une fois l'information récupérée, elle est stockée dans Redis.**

- **2ème GET (Source Redis) : La réponse est quasi instantanée. L'application récupère la donnée directement depuis la mémoire vive (RAM) de Redis sans solliciter la base de données principale.**

Q4/Pourquoi invalide-t-on le cache Redis après un POST /commande

On invalide (supprime) le cache pour garantir la cohérence des données. Si un utilisateur crée une nouvelle commande via un `POST`, les données stockées dans MongoDB changent. Si on ne supprimait pas le cache Redis, le prochain `GET` renverrait l'ancienne liste des commandes (sans la nouvelle) car Redis ne saurait pas que la base de données a été mise à jour. En supprimant le cache, on force le système à refaire un `GET` propre sur MongoDB lors de la prochaine demande.

Q5/En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ améliore la résilience grâce au découplage asynchrone :

1. **Isolation des pannes : Si le service de notification (consommateur) tombe en panne, le service de commande (producteur) peut continuer à fonctionner. Les messages ne sont pas perdus, ils sont stockés en sécurité dans la file d'attente (Queue).**
2. **Reprise après sinistre : Dès que le service de notification redémarre, il consomme tous les messages accumulés dans RabbitMQ. Le système "rattrape" son retard sans perte de données**.

Q6/Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dansnginx.conf ?

Pour ajouter une instance (par exemple `app3` sur le port `3003`), il faudrait modifier le bloc `upstream` dans le fichier `nginx.conf`.

TP 4 – Architecture Orientée Services (SAO)

1. Introduction :

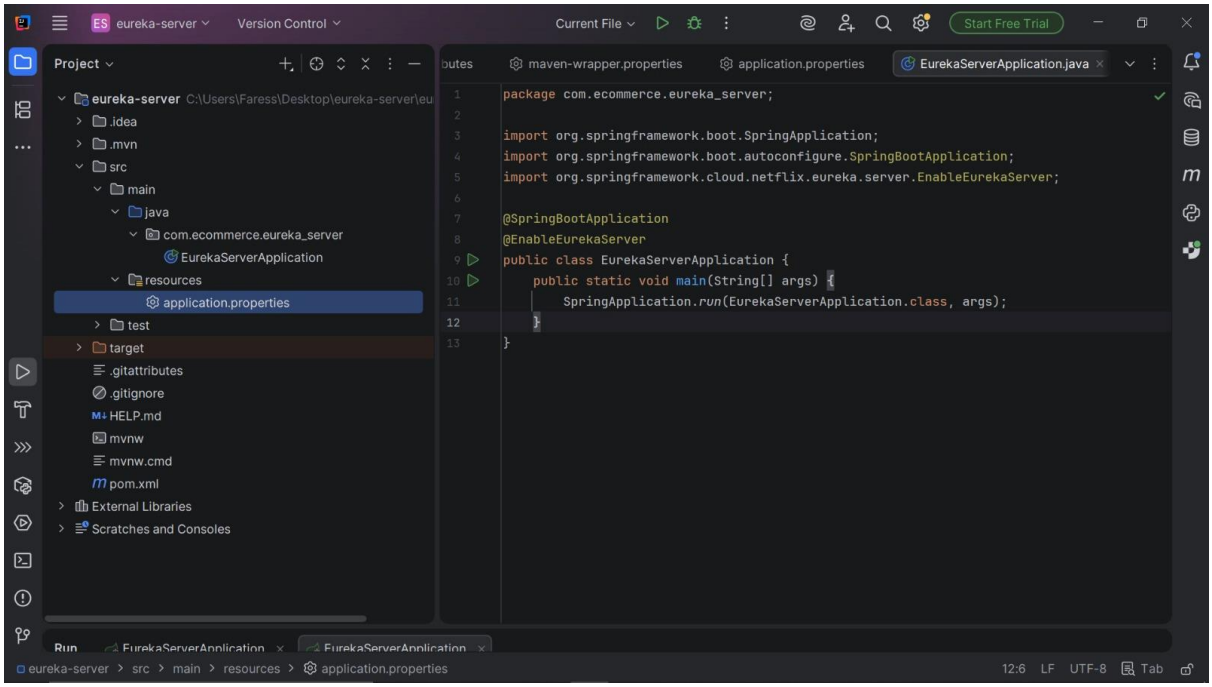
L'architecture orientee services (SOA - Service-Oriented Architecture) est un paradigme de conception logicielle qui permet de structurer une application en services independants et reutilisables. Dans ce TP, nous avons cree une application e-commerce utilisant Spring Boot pour implementer une architecture SOA.

SOA represente une evolution naturelle par rapport au monolithe (TP1/TP2) : au lieu d'avoir une seule application qui fait tout, on decoupe le systeme en services independants qui communiquent entre eux via des APIs REST.

2. Objectifs Pédagogiques :

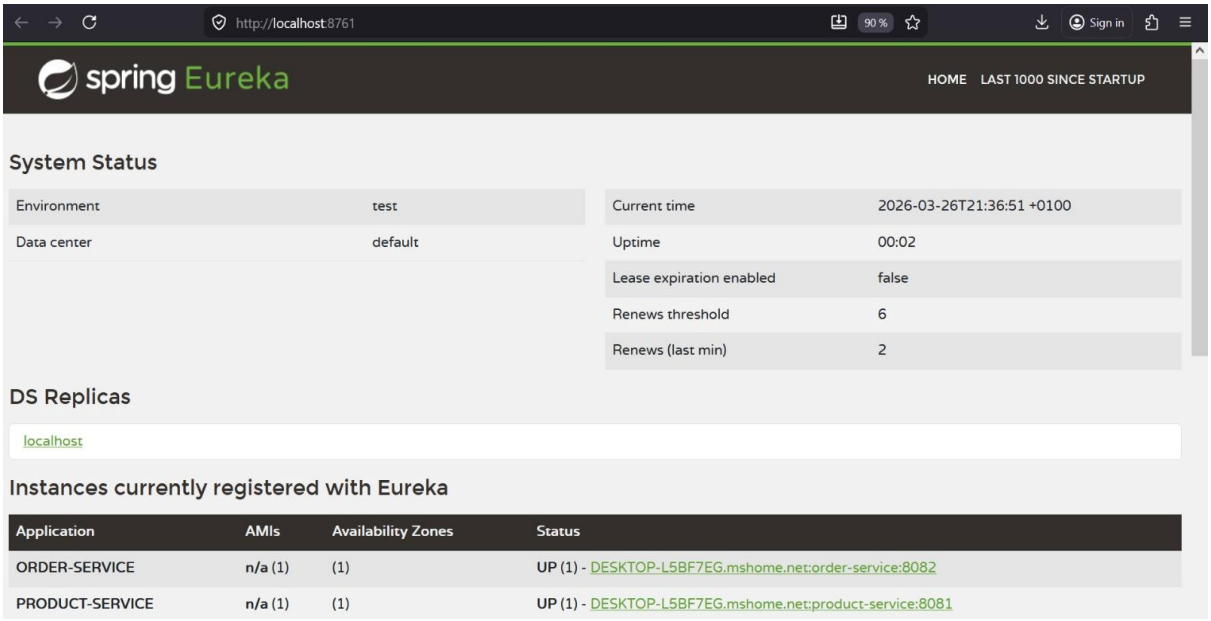
- Comprendre les principes de l'architecture SOA
- Creer des microservices avec Spring Boot
- Implementer la communication inter-services via REST (OpenFeign)
- Utiliser un service de decouverte Eureka pour l'enregistrement des services
- Gerer une base de donnees avec Spring Data JPA (H2 en memoire)

3. Etape 1 - Eureka Server :

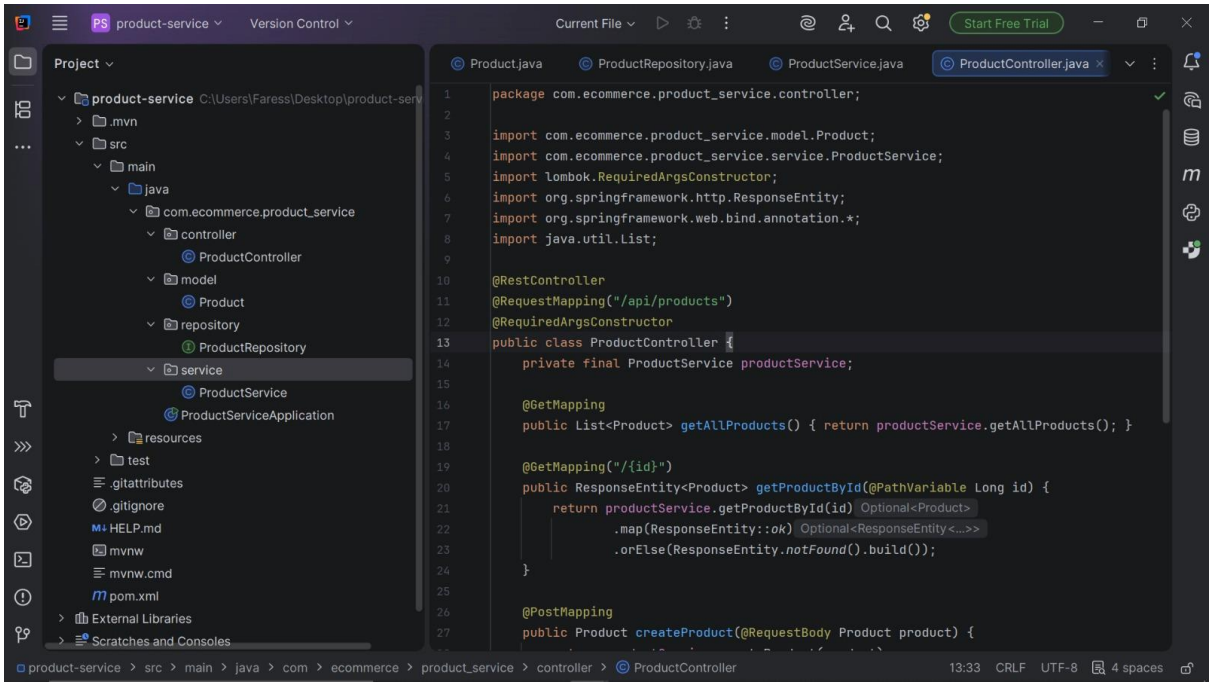


3.1 Tableau de bord Eureka :

Apres démarrage, le tableau de bord Eureka est accessible sur http://localhost:8761. Il affiche tous les services enregistres avec leur statut (UP/DOWN), leur adresse et leur port. On peut voir PRODUCT-SERVICE et ORDER-SERVICE enregistres avec le statut UP.



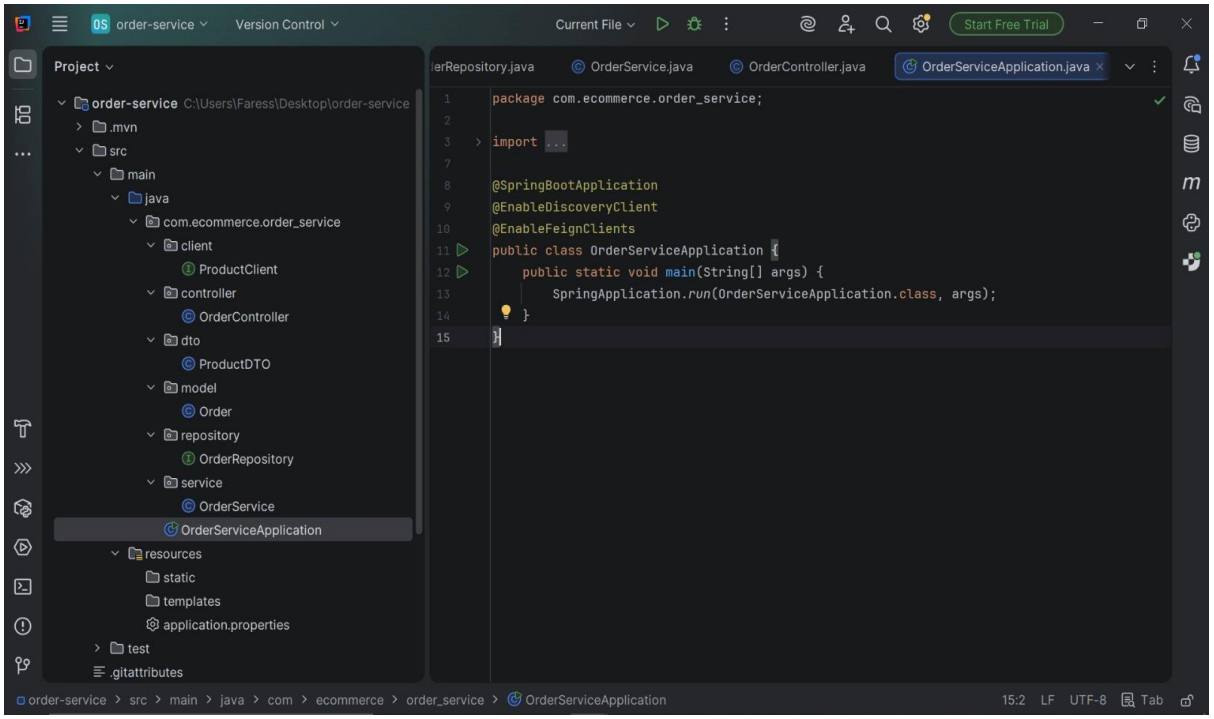
4. Etape 2 - Product Service :



5. Etape 3 - Order Service

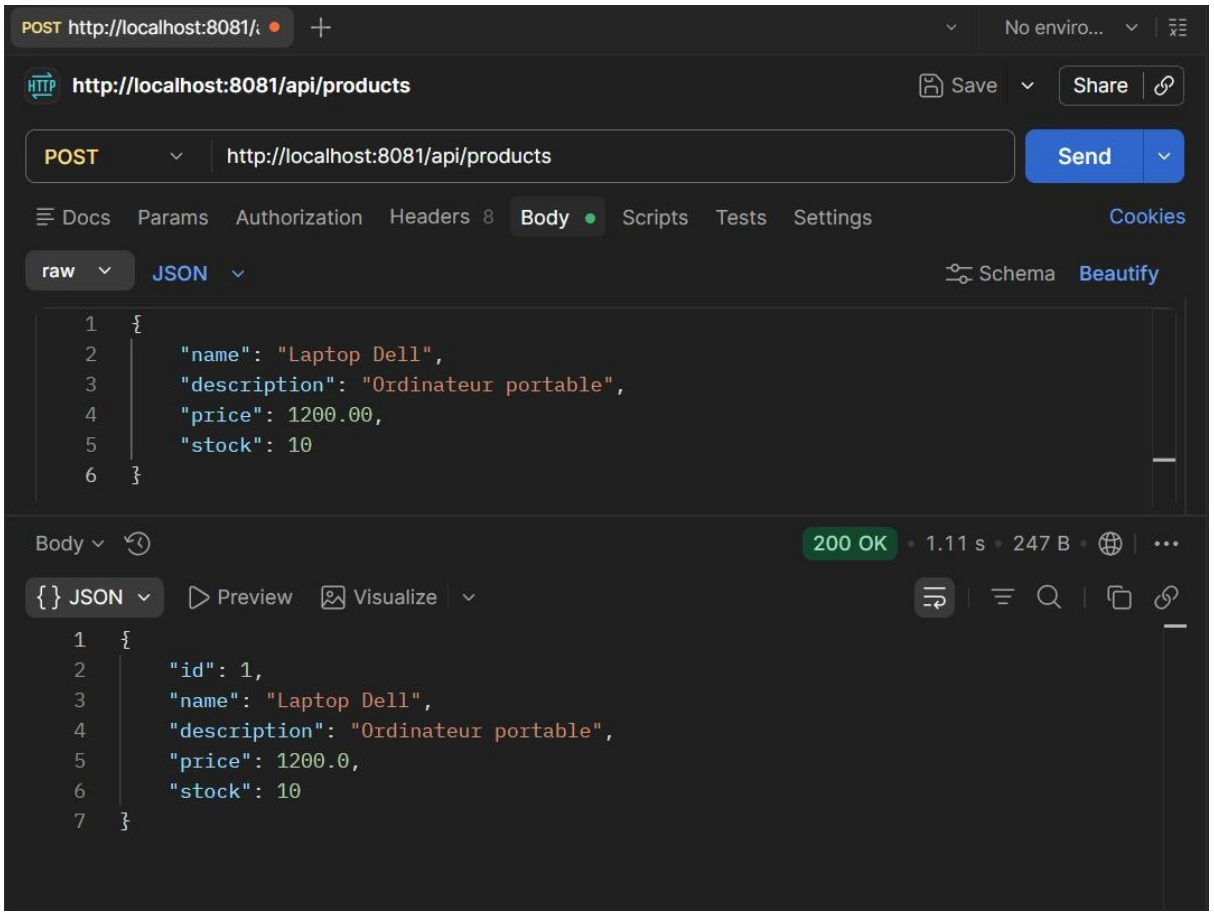
5.1 OpenFeign :

OpenFeign est un client HTTP declaratif de Spring Cloud. Il permet d'appeler un autre microservice comme si c'etait une methode Java locale, sans ecrire de code HTTP manuellement. OpenFeign utilise Eureka pour trouver l'adresse de Product Service automatiquement.



6. Tests avec Postman :

6.1 Test 1 :



6.2 Test 2:j

GET http://localhost:8081/a | GET http://localhost:8081/a | No enviro...

http://localhost:8081/api/products | Save | Share

GET | http://localhost:8081/api/products | Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK | 17 ms | 249 B

{} JSON | Preview | Visualize


```
[
    {
        "id": 1,
        "name": "Laptop Dell",
        "description": "Ordinateur portable",
        "price": 1200.0,
        "stock": 10
    }
]
```


http://localhost:8082/api/orders | Save | Share

POST | http://localhost:8082/api/orders?productId=1&quantity=2 | Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

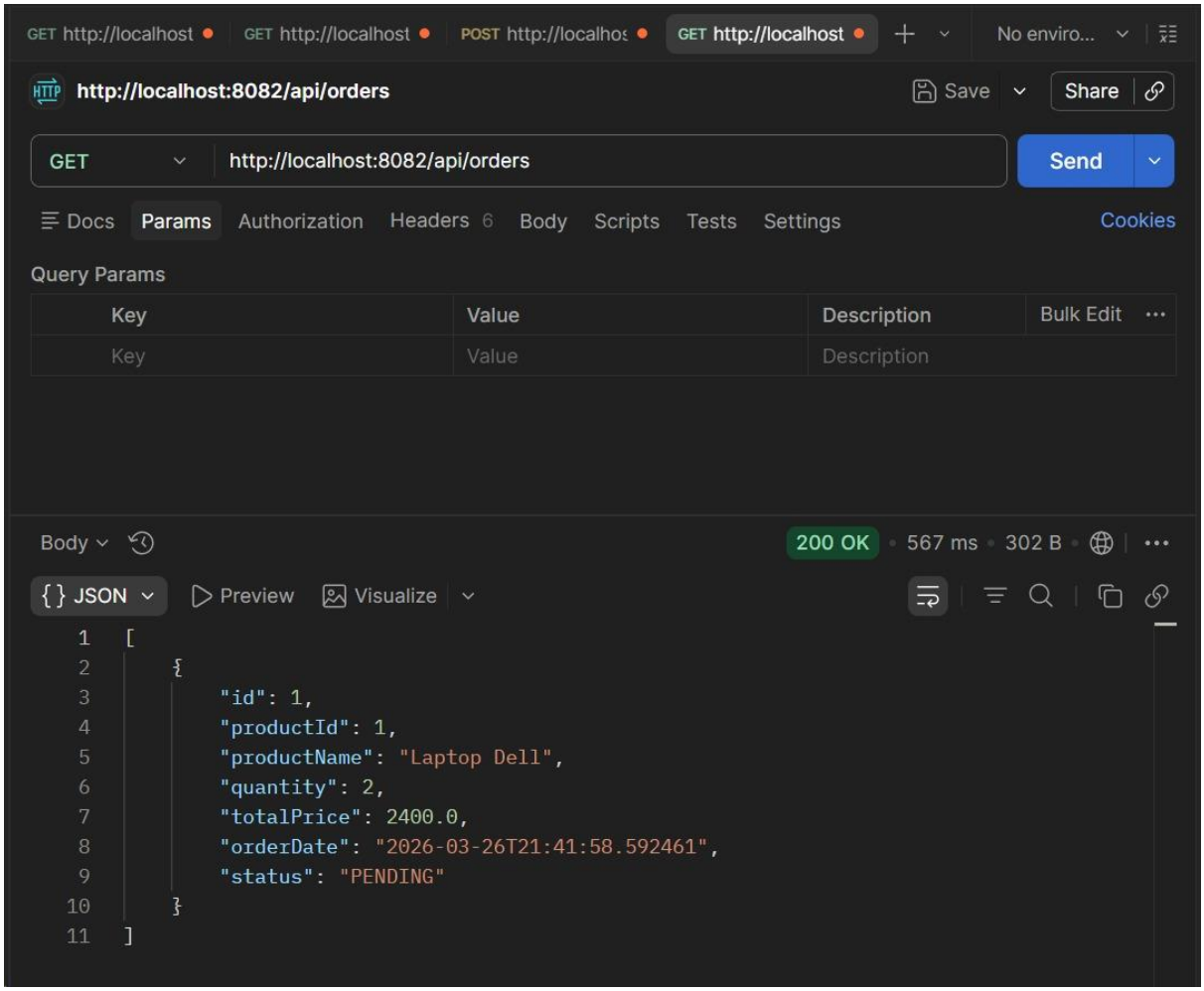
Key	Value	Description	Bulk Edit
✓ productId	1		
✓ quantity	2		
Key	Value	Description	

Body | 200 OK | 1.54 s | 301 B

{} JSON | Preview | Visualize


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop Dell",
    "quantity": 2,
    "totalPrice": 2400.0,
    "orderDate": "2026-03-26T21:41:58.5924606",
    "status": "PENDING"
}
```

6.4 Test 4:



11. Conclusion:

Le TP4 a introduit l'architecture SOA avec Spring Boot, Eureka et OpenFeign. Cette architecture represente une evolution majeure par rapport au monolithe des TPs precedents.

Les avantages principaux sont l'indépendance des services (chaque service peut etre déployé, mis a jour ou scalie indépendamment), l'isolation des pannes (si Order Service tombe, Product Service continue de fonctionner), et la scalabilité sélective (on peut lancer plusieurs instances de Product Service si nécessaire).

Fin